

Week 3 Module:

Date: 29/06/2026

Question 1 (Easy):

A university's Training and Placement Cell have successfully completed its annual placement drive. The placement department has collected the salary packages (in LPA) offered to students by different companies. However, the records were entered in the order they were received, resulting in an unsorted list of package values.

Before publishing the placement statistics report, the department wants all package values arranged in ascending order. Since the university plans to handle placement data for thousands of students in the coming years, they want a sorting technique that is efficient for large datasets.

Your task is to implement the **Merge Sort algorithm** to sort the package values in increasing order. Merge Sort follows a divide-and-conquer approach by dividing the dataset into smaller subarrays, sorting them recursively, and then merging them back together.

The final sorted list will help the university identify minimum, median, and maximum packages while generating analytical reports.

Input

- First line contains an integer **N**, representing the number of placed students.
- Second line contains **N space-separated integers**, representing package values (multiplied by 100 to avoid decimals).

Output

Print the package values in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $100 \leq \text{Package} \leq 10000$

Example

Input:

6
450 1200 800 650 300 1500

Output:

300 450 650 800 1200 1500

Question 2 (Medium):

A large e-commerce company tracks the time taken (in minutes) to process customer orders before shipment. Due to varying warehouse workloads, processing times differ significantly from one order to another.

The analytics team wants to study operational efficiency by sorting all processing times and generating statistical information. Since millions of orders are processed every month, the company requires an efficient sorting technique with predictable performance.

Your task is to use **Merge Sort** to arrange all processing times in ascending order.

After sorting, perform the following operations:

1. Display the sorted processing times.
2. Find the **median processing time**.
3. Count the number of orders whose processing time is greater than the median.
4. Display the difference between the fastest and slowest processing time.

The solution should efficiently handle large datasets and demonstrate a clear understanding of Merge Sort's divide-and-conquer strategy.

Input

- First line contains integer **N**.
- Second line contains **N space-separated processing times**.

Output

- Sorted processing times.
- Median processing time.
- Count of orders greater than median.
- Difference between maximum and minimum processing time.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Time} \leq 100000$

Example

Input:

```
7
12 25 18 30 15 22 40
```

Output:

```
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
```

Topic 2: Quick Sort

Question 1 (Easy):

A cloud infrastructure company manages hundreds of servers distributed across different geographical locations. Every hour, each server reports its average response time in milliseconds. The collected data is stored in the order it arrives and is not sorted.

To generate performance reports, the operations team wants the response times arranged from the fastest server to the slowest server. Since the data size can become very large, an efficient in-place sorting technique is required.

Your task is to implement **Quick Sort** to sort the response times in ascending order. Built-in sorting methods are not allowed.

After sorting, the operations team will use the report to identify high-latency servers and optimize system performance.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

Print the response times in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Response Time} \leq 10000$

Example

Input:

6
120 45 78 200 60 95

Output:

45 60 78 95 120 200

Question 2 (Medium):

A financial analytics company receives daily trade-value data from multiple stock exchanges. Analysts use this information to identify high-value trading activity and market trends.

The trade values are received in random order throughout the day. Before analysis can begin, the data must be sorted in descending order.

Your task is to implement **Quick Sort** to arrange the trade values from highest to lowest.

After sorting, perform the following operations:

1. Display the sorted trade values.
2. Find the **Top 5 highest trade values**.
3. Calculate the average of the Top 5 values.
4. Count how many trade values are greater than the overall average of all trade values.

This problem evaluates your understanding of partitioning, recursion, and real-world data analysis using Quick Sort.

Input

- First line contains integer **N**.
- Second line contains **N space-separated trade values**.

Output

- Sorted trade values (descending).
- Top 5 values.
- Average of Top 5 values.
- Count of values greater than overall average.

Constraints

- $5 \leq N \leq 100000$
- $1 \leq \text{Trade Value} \leq 10^9$

Example

Input:

8
500 1200 700 2000 900 1500 3000 2500

Output:

3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 4

Topic 3: Heap Sort

Question 1 (Easy):

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed $O(N \log N)$ performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

Input

- First line contains integer **N**.
- Second line contains **N space-separated player scores**.

Output

Print the scores in ascending order.

Constraints

- $1 \leq N \leq 100000$
- $0 \leq \text{Score} \leq 1000000$

Example

Input:

6
850 1200 950 600 1500 1000

Output:

600 850 950 1000 1200 1500

Question 2 (Medium):

A multi-specialty hospital maintains records of emergency response times for all critical cases handled during a month. Each response time represents the number of minutes taken by the emergency team to reach and begin treatment for a patient.

Hospital administrators want to analyze the efficiency of their emergency services. To do so, all response times must first be sorted using **Heap Sort**.

After sorting the response times in ascending order, generate the following statistics:

1. Display the sorted response times.
2. Find the fastest response time.
3. Find the slowest response time.
4. Calculate the average response time.
5. Count the number of cases handled faster than the average.
6. Calculate the percentage of cases handled faster than the average.

The hospital intends to use this information to improve resource allocation and reduce emergency response delays.

Input

- First line contains integer **N**.
- Second line contains **N space-separated response times**.

Output

- Sorted response times.
- Fastest response time.
- Slowest response time.
- Average response time.
- Number of cases faster than average.
- Percentage of cases faster than average.

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{Time} \leq 500$

Example

Input:

8
12 7 15 9 20 5 10 8

Output:

5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%

① Algo

- 1) Read the no. of elements N .
- 2) Read the array elements.
- 3) Call mergeSort (arr, 0, $N-1$);
- 4) In mergeSort():
 - if left < right:
 - find the middle index
 - Recursively sort the left half.
 - Recursively sort the right half.
 - Merge both sorted halves using merge().

5) In merge():

- Create 2 temp array for left & right halves.
- Copy the elements into temp arrays.
- Compare elements from both arrays & place smaller element into original array.
- Copy remaining elements from left or right array.

6) Print the sorted array.

② Algo

- 1) Read the no. of elements N
- 2) Read all array elements.
- 3) Sort the array using merge sort
- 4) Print the sorted array.
- 5) Find the median:
 - If ~~odd~~ N is odd, median = middle element
 - If N is even, median = avg of 2 middle element
- 6) Traverse the array and count the no. of elements greater than median.
- 7) Find the diff. b/w largest & smallest element
- 8) Print: sorted array, median, elements greater than median, difference,

(3)

Algo

- 1) Read no. of elements N .
- 2) Read all array elements.
- 3) Call quick sort ($arr, 0, N-1$).
- 4) In quick sort ($low, high$):
 - find pivot pos. using partition ($low, high$).
 - Recursively sort left subarray.
 - Recursively sort right subarray.

5) In partition ($low, high$),

- select last element as pivot.
- initialize $i = low - 1$.
- Traverse array from low to $high - 1$.
- If current element $\leq$ pivot:
 - increment i .
 - swap ($arr[i], arr[high]$).
- After traversal, swap pivot with $arr[i+1]$.
- Return pivot index.

6) Print sorted array.

(4)

Algo

- 1) Read no. of elements N .
- 2) Read all trade values & calculate their total sum.
- 3) Sort array in desc. order using quick sort.
- 4) In partition ($low, high$):
 - select last element as pivot.
 - initialize $i = low - 1$.
 - Traverse array from low to $high - 1$.
 - If current element $\geq$ pivot:
 - increment i .
 - swap ($arr[i], arr[high]$).

- Place the pivot at correct pos.
- 5) Print the sorted array.
 - 6) Print first 5 elements or top 5 values.
 - 7) Calculate avg of top 5 values.
 - 8) Calculate overall avg of all trade values.
 - 9) Traverse array and count the values greater than overall avg.
 - 10) Print: sorted array, Top 5 values, avg of top 5, no. > overall avg.

⑤ Algo

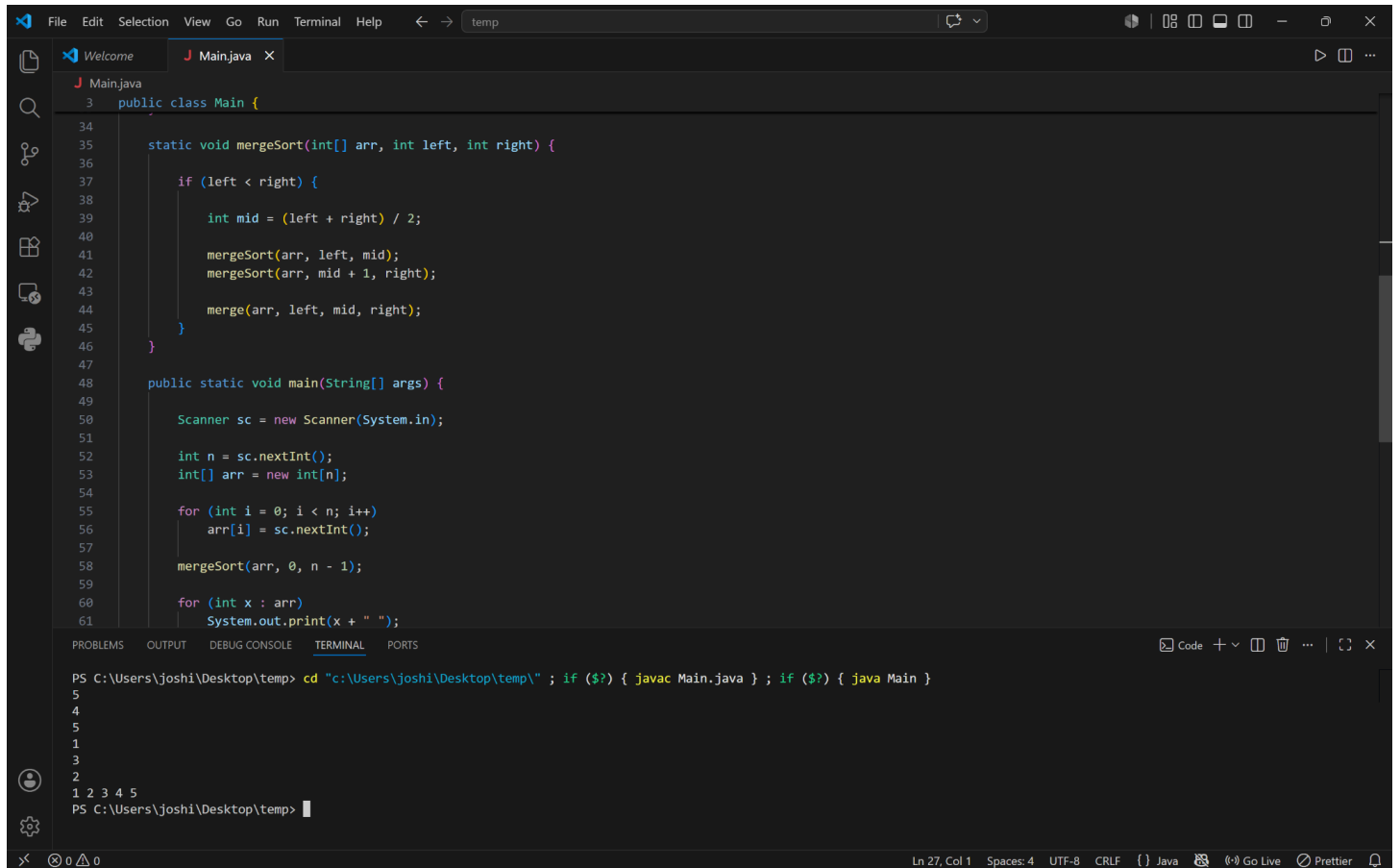
- 1) Read no. of elements N .
- 2) Read all array elements.
- 3) Build a max heap from array.
- 4) Repeat until only one element remains in the heap.
 - swap the root with last element.
 - Reduce heap size by 1.
 - Apply heapify() on root to restore max heap property.
- 5) After all ~~the~~ iterations, array will be sorted in asc. order.
- 6) Print the sorted array.

⑥ Algo

- 1) Read the no. of elements N .
 - 2) Read all response times and calculate their total sum.
 - 3) Sort the array using heap sort.
 - 4) Print sorted array.
 - 5) Find fastest response time. (first element)
 - 6) Find slowest response time (last element).
 - 7) Calculate avg response time
- $$\text{Avg} = \frac{\text{Sum}}{N}$$

- 8) Traverse the sorted array and count no. of response times less than avg.
- 9) calculate % of cases handled faster than avg
using: $\text{percentage} = (\text{count} \times 100) / N$.
- 10) Print: sorted response times, fastest one, slowest one, avg response time, cases faster than avg, % of cases faster than avg.

Q1 - Merge Sort (Package Values Sorting)



The image shows a screenshot of an IDE (IntelliJ IDEA) with a Java file named `Main.java` open. The code implements a Merge Sort algorithm. The `mergeSort` method is a static method that takes an array `arr` and indices `left` and `right` as parameters. It recursively splits the array into two halves, sorts each half, and then merges them back together. The `main` method uses a `Scanner` to read input from the user, creates an array of size `n`, and then calls `mergeSort` to sort the array. The output of the program is displayed in the terminal window at the bottom.

```
3 public class Main {
34
35     static void mergeSort(int[] arr, int left, int right) {
36
37         if (left < right) {
38
39             int mid = (left + right) / 2;
40
41             mergeSort(arr, left, mid);
42             mergeSort(arr, mid + 1, right);
43
44             merge(arr, left, mid, right);
45         }
46     }
47
48     public static void main(String[] args) {
49
50         Scanner sc = new Scanner(System.in);
51
52         int n = sc.nextInt();
53         int[] arr = new int[n];
54
55         for (int i = 0; i < n; i++)
56             arr[i] = sc.nextInt();
57
58         mergeSort(arr, 0, n - 1);
59
60         for (int x : arr)
61             System.out.print(x + " ");
```

The terminal window shows the following commands and output:

```
PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
5
4
5
1
3
2
1 2 3 4 5
PS C:\Users\joshi\Desktop\temp>
```

FileEditSelectionViewGoRunTerminalHelptemp

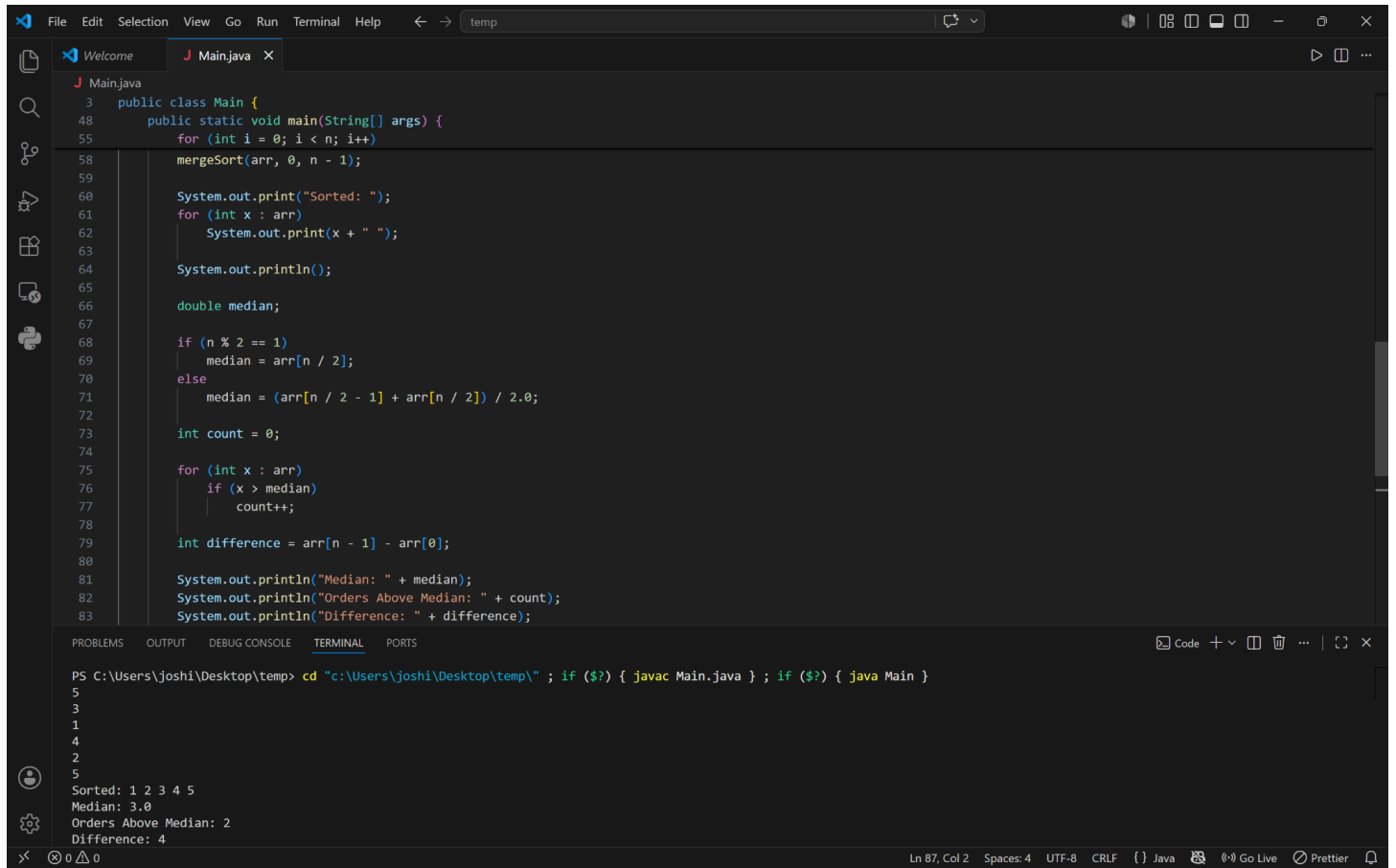
WelcomeMain.java

Main.java

```
3 public class Main {
4
5     static void merge(int[] arr, int left, int mid, int right) {
6
7         int n1 = mid - left + 1;
8         int n2 = right - mid;
9
10        int[] L = new int[n1];
11        int[] R = new int[n2];
12
13        for (int i = 0; i < n1; i++)
14            L[i] = arr[left + i];
15
16        for (int j = 0; j < n2; j++)
17            R[j] = arr[mid + 1 + j];
18
19        int i = 0, j = 0, k = left;
20
21        while (i < n1 && j < n2) {
22            if (L[i] <= R[j])
23                arr[k++] = L[i++];
24            else
25                arr[k++] = R[j++];
26        }
27
28        while (i < n1)
29            arr[k++] = L[i++];
30
31        while (j < n2)
32            arr[k++] = R[j++];
33    }
34
35    static void mergeSort(int[] arr, int left, int right) {
36
37        if (left < right) {
38
39            int mid = (left + right) / 2;
40
41            mergeSort(arr, left, mid);
42            mergeSort(arr, mid + 1, right);
43        }
44    }
45}
```

Ln 65, Col 2 Spaces: 4 UTF-8 CRLF Java Go Live Prettier

Q2 - Merge Sort (Processing Time Analysis)



The screenshot shows an IDE with a Java file named `Main.java` and a terminal window. The Java code implements a Merge Sort algorithm and calculates the median, the number of elements above the median, and the difference between the last and first elements of the sorted array.

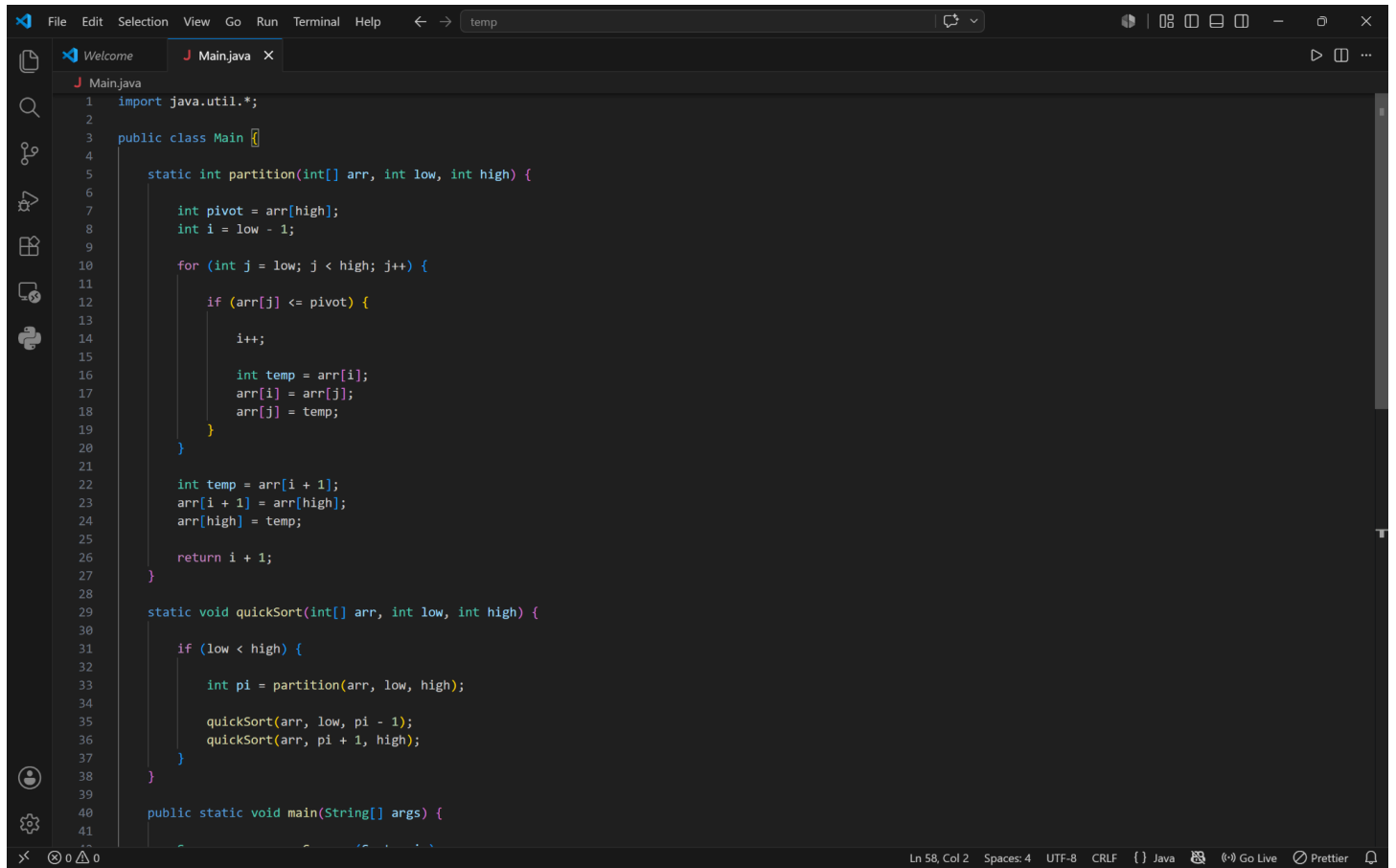
```
3 public class Main {
48     public static void main(String[] args) {
55         for (int i = 0; i < n; i++)
58             mergeSort(arr, 0, n - 1);
59
60         System.out.print("Sorted: ");
61         for (int x : arr)
62             System.out.print(x + " ");
63
64         System.out.println();
65
66         double median;
67
68         if (n % 2 == 1)
69             median = arr[n / 2];
70         else
71             median = (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
72
73         int count = 0;
74
75         for (int x : arr)
76             if (x > median)
77                 count++;
78
79         int difference = arr[n - 1] - arr[0];
80
81         System.out.println("Median: " + median);
82         System.out.println("Orders Above Median: " + count);
83         System.out.println("Difference: " + difference);
84     }
85 }
```

The terminal window shows the command to compile and run the program, followed by the output:

```
PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
5
3
1
4
2
5
Sorted: 1 2 3 4 5
Median: 3.0
Orders Above Median: 2
Difference: 4
```

The status bar at the bottom indicates the cursor is at line 87, column 2, with 4 spaces, UTF-8 encoding, CRLF line endings, and the Java language selected. It also shows icons for Go Live and Prettier.

Q3 - Quick Sort (Server Response Time Sorting)



The image shows a screenshot of an IDE (Integrated Development Environment) with a dark theme. The main editor window displays a Java file named `Main.java`. The code implements a Quick Sort algorithm. The `partition` method is a static method that takes an array `arr`, a low index, and a high index. It selects the last element as the pivot, moves elements less than or equal to the pivot to the left, and returns the pivot's final position. The `quickSort` method is also static and recursively calls `partition` and itself to sort the array. The `main` method is a standard entry point for a Java application. The IDE interface includes a menu bar (File, Edit, Selection, View, Go, Run, Terminal, Help), a toolbar with icons for file operations, a sidebar with icons for project structure, search, and other tools, and a status bar at the bottom showing the current line and column (Ln 58, Col 2), encoding (UTF-8), line endings (CRLF), and active plugins (Java, Go Live, Prettier).

```
1  import java.util.*;
2
3  public class Main {
4
5      static int partition(int[] arr, int low, int high) {
6
7          int pivot = arr[high];
8          int i = low - 1;
9
10         for (int j = low; j < high; j++) {
11
12             if (arr[j] <= pivot) {
13
14                 i++;
15
16                 int temp = arr[i];
17                 arr[i] = arr[j];
18                 arr[j] = temp;
19             }
20
21             int temp = arr[i + 1];
22             arr[i + 1] = arr[high];
23             arr[high] = temp;
24
25             return i + 1;
26         }
27     }
28
29     static void quickSort(int[] arr, int low, int high) {
30
31         if (low < high) {
32
33             int pi = partition(arr, low, high);
34
35             quickSort(arr, low, pi - 1);
36             quickSort(arr, pi + 1, high);
37         }
38     }
39
40     public static void main(String[] args) {
41
42         // ... (code continues) ...
43     }
44 }
```


FileEditSelectionViewGoRunTerminalHelptemp

WelcomeMain.java

3public class Main {
29static void quickSort(int[] arr, int low, int high) {
31if (low < high) {
34
35quickSort(arr, low, pi - 1);
36quickSort(arr, pi + 1, high);
37}
38}
39
40public static void main(String[] args) {
41
42Scanner sc = new Scanner(System.in);
43
44int n = sc.nextInt();
45
46int[] arr = new int[n];
47
48for (int i = 0; i < n; i++)
49arr[i] = sc.nextInt();
50
51quickSort(arr, 0, n - 1);
52
53for (int x : arr)
54System.out.print(x + " ");
55
56sc.close();

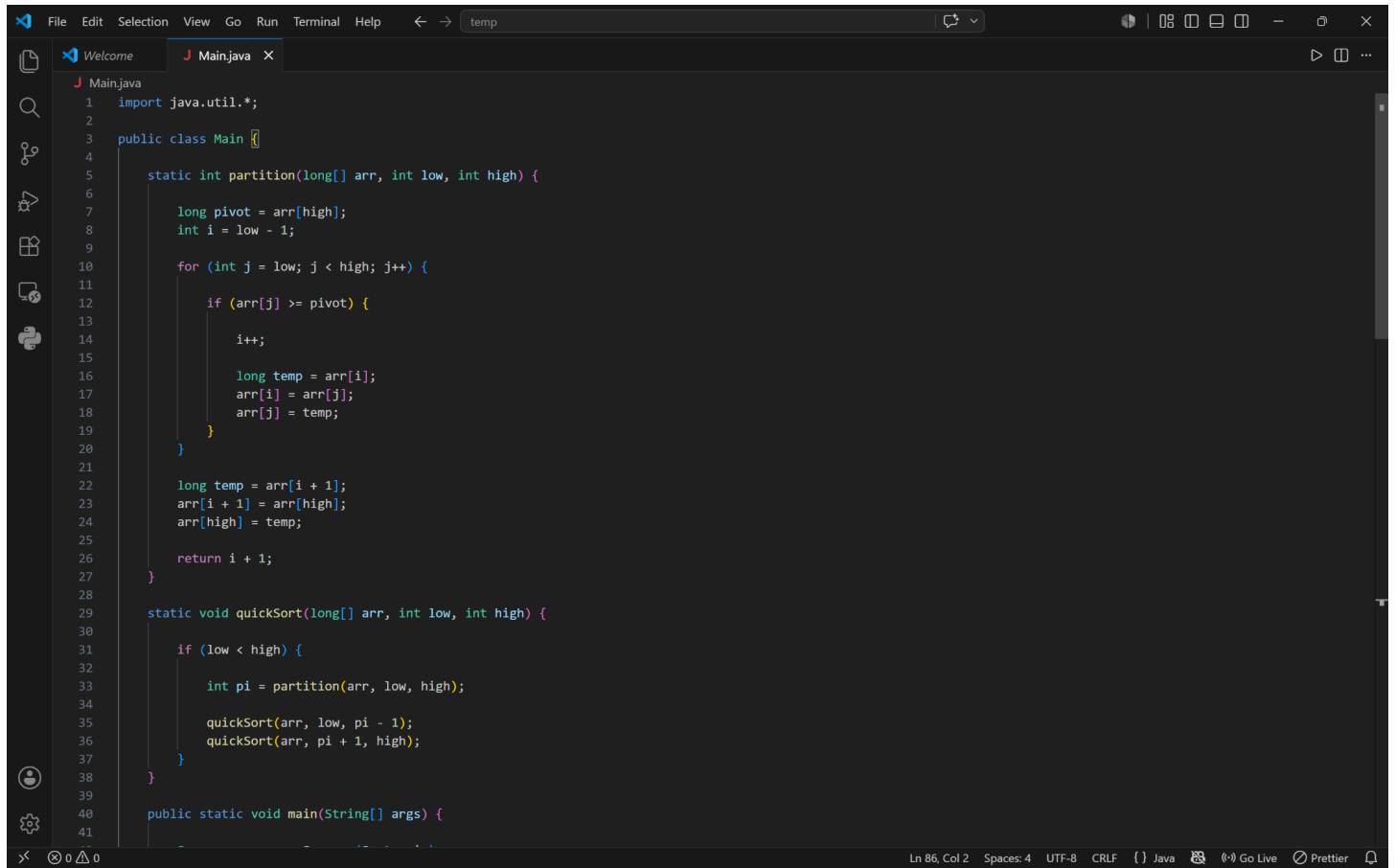
PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

Code + -

PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp" ; if (\$?) { javac Main.java } ; if (\$?) { java Main }
5
2
3
4
1
5
1 2 3 4 5
PS C:\Users\joshi\Desktop\temp>

Ln 58, Col 2Spaces: 4UTF-8CRLFJavaGo LivePrettier

Q4 - Quick Sort (Trade Value Analysis)



The image shows a screenshot of an IDE with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The toolbar shows icons for undo, redo, and search. The editor has two tabs: 'Welcome' and 'Main.java'. The 'Main.java' tab is active, showing the following code:

```
1  import java.util.*;
2
3  public class Main {
4
5      static int partition(long[] arr, int low, int high) {
6
7          long pivot = arr[high];
8          int i = low - 1;
9
10         for (int j = low; j < high; j++) {
11
12             if (arr[j] >= pivot) {
13
14                 i++;
15
16                 long temp = arr[i];
17                 arr[i] = arr[j];
18                 arr[j] = temp;
19             }
20
21             long temp = arr[i + 1];
22             arr[i + 1] = arr[high];
23             arr[high] = temp;
24
25             return i + 1;
26         }
27     }
28
29     static void quickSort(long[] arr, int low, int high) {
30
31         if (low < high) {
32
33             int pi = partition(arr, low, high);
34
35             quickSort(arr, low, pi - 1);
36             quickSort(arr, pi + 1, high);
37         }
38     }
39
40     public static void main(String[] args) {
41
```

The status bar at the bottom shows 'Ln 86, Col 2', 'Spaces: 4', 'UTF-8', 'CRLF', 'Java', 'Go Live', 'Prettier', and a bell icon.

FileEditSelectionViewGoRunTerminalHelptemp

WelcomeMain.java

3public class Main {
40public static void main(String[] args) {
54quickSort(arr, 0, n - 1);
55
56System.out.print("Sorted: ");
57for (long x : arr)
58System.out.print(x + " ");
59
60System.out.println();
61
62long topSum = 0;
63
64System.out.print("Top 5: ");
65for (int i = 0; i < 5; i++) {
66System.out.print(arr[i] + " ");
67topSum += arr[i];
68}
69
70System.out.println();
71
72double topAverage = topSum / 5.0;
73double overallAverage = sum / (double) n;
74
75int count = 0;
76
77for (long x : arr)
78if (x > overallAverage)

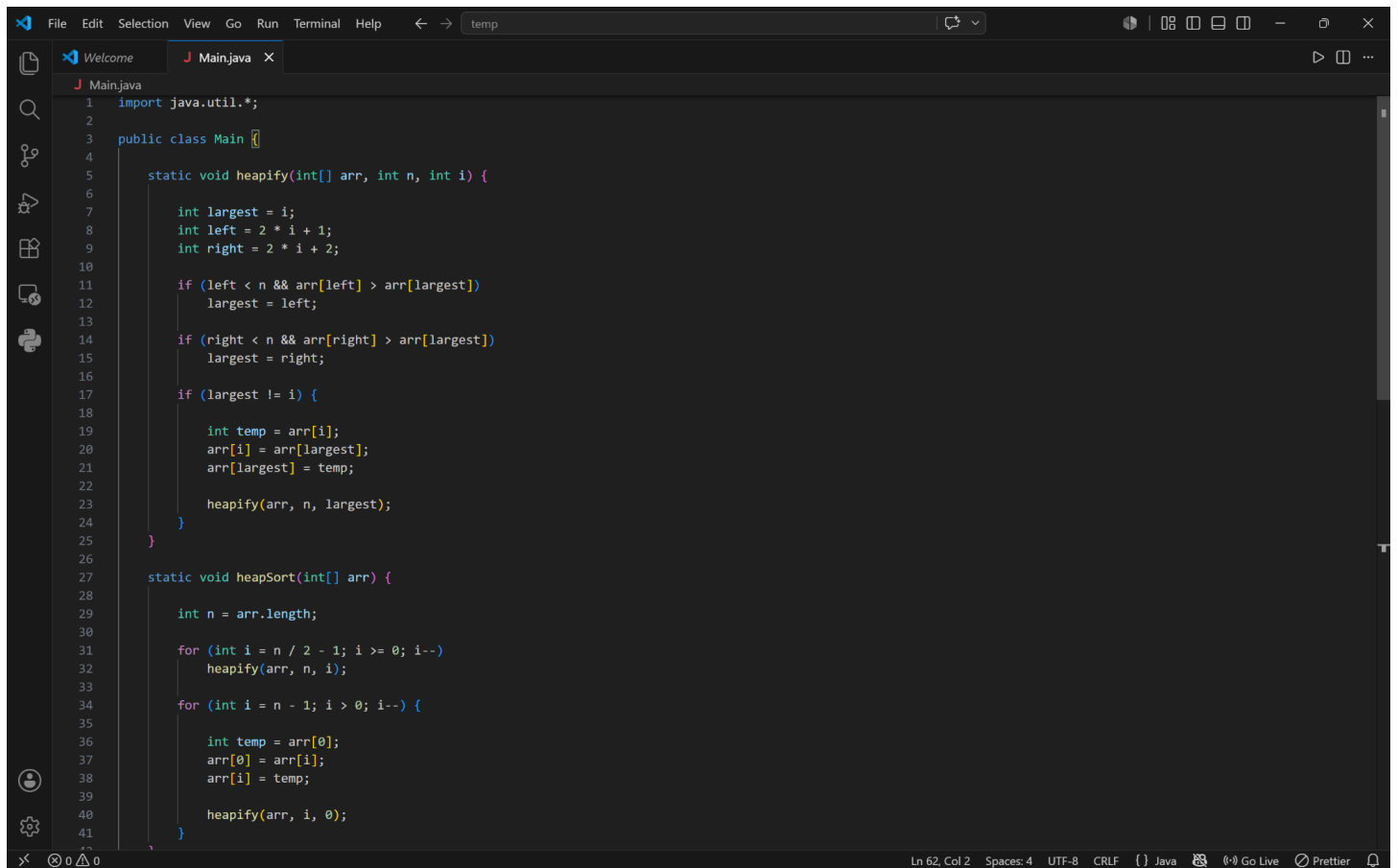
PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

Code+−□□□□□□□□

PS C:\Users\joshi\Desktop\temp> cd "c:\Users\joshi\Desktop\temp\" ; if (\$?) { javac Main.java } ; if (\$?) { java Main }
5
1
3
2
5
4
Sorted: 5 4 3 2 1
Top 5: 5 4 3 2 1
Average of Top 5: 3.0
Values Above Overall Average: 2
PS C:\Users\joshi\Desktop\temp>

Ln 86, Col 2Spaces: 4UTF-8CRLF() JavaGo LivePrettier

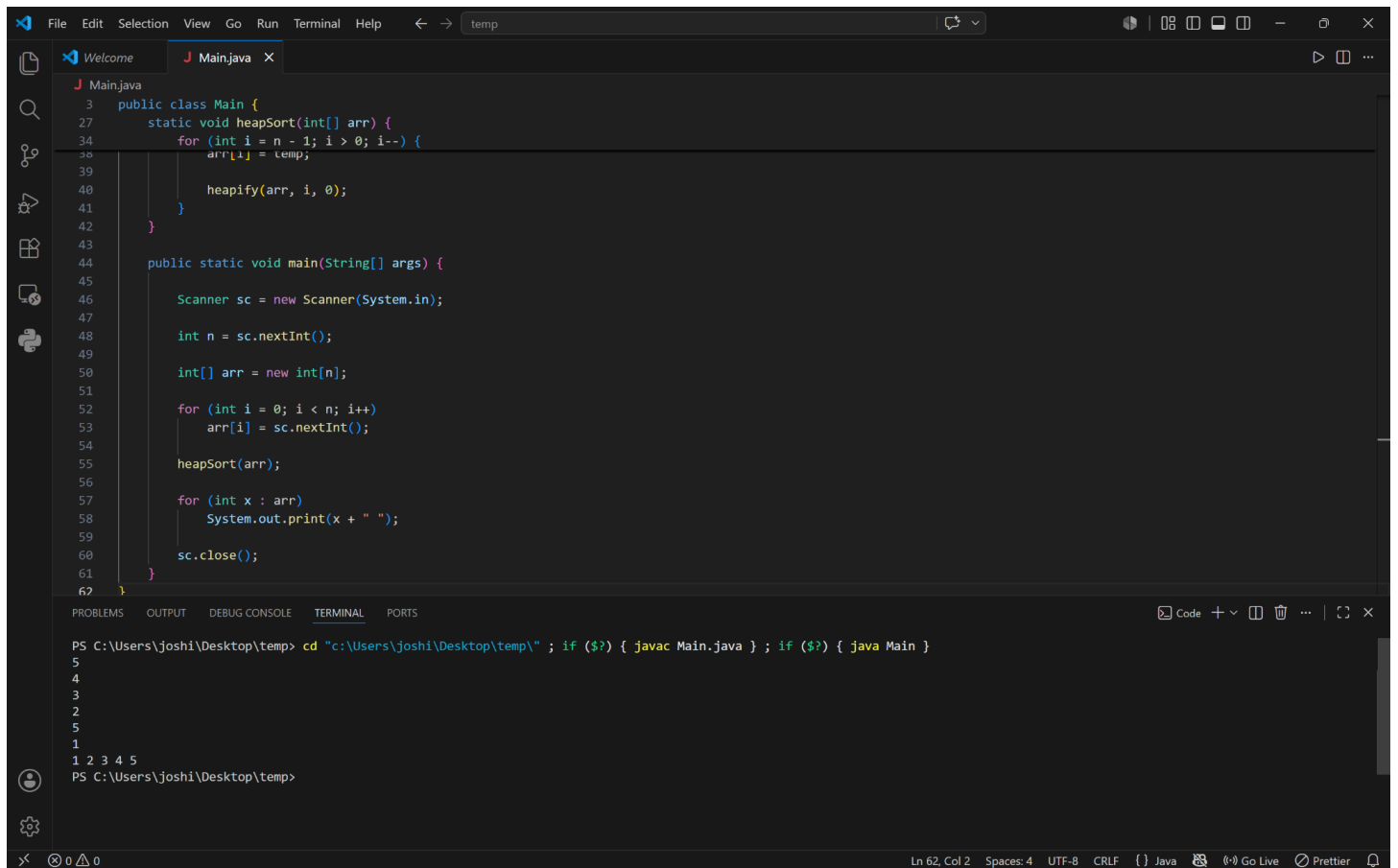
Q5 - Heap Sort (Player Score Sorting)



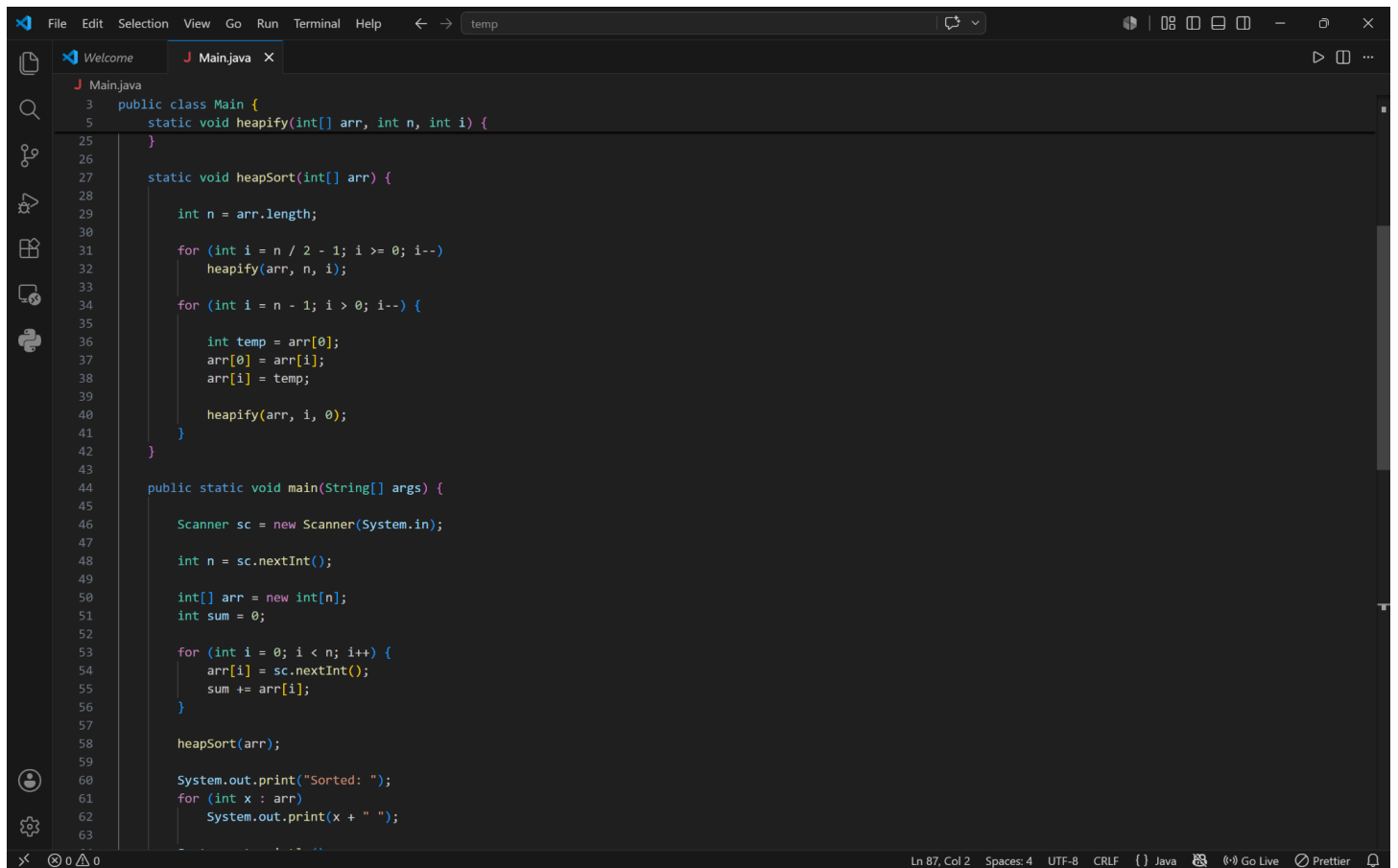
The image shows a screenshot of an IDE with a dark theme. The main editor window displays a Java program for Heap Sort. The code is as follows:

```
1 import java.util.*;
2
3 public class Main {
4
5     static void heapify(int[] arr, int n, int i) {
6
7         int largest = i;
8         int left = 2 * i + 1;
9         int right = 2 * i + 2;
10
11         if (left < n && arr[left] > arr[largest])
12             largest = left;
13
14         if (right < n && arr[right] > arr[largest])
15             largest = right;
16
17         if (largest != i) {
18
19             int temp = arr[i];
20             arr[i] = arr[largest];
21             arr[largest] = temp;
22
23             heapify(arr, n, largest);
24         }
25     }
26
27     static void heapSort(int[] arr) {
28
29         int n = arr.length;
30
31         for (int i = n / 2 - 1; i >= 0; i--)
32             heapify(arr, n, i);
33
34         for (int i = n - 1; i > 0; i--) {
35
36             int temp = arr[0];
37             arr[0] = arr[i];
38             arr[i] = temp;
39
40             heapify(arr, i, 0);
41         }
42     }
43 }
```

The IDE interface includes a menu bar (File, Edit, Selection, View, Go, Run, Terminal, Help), a toolbar with icons for file operations, a sidebar with icons for project structure, search, and other tools, and a status bar at the bottom showing 'Ln 62, Col 2', 'Spaces: 4', 'UTF-8', 'CRLF', 'Java', 'Go Live', 'Prettier', and a bell icon.



Q6 - Heap Sort (Hospital Response Time Analysis)



The screenshot shows an IDE window with a Java file named 'Main.java'. The code implements a Heap Sort algorithm. It includes a 'heapify' method to maintain the heap property, a 'heapSort' method to sort the array, and a 'main' method to take input and print the sorted array. The code is as follows:

```
3 public class Main {
5     static void heapify(int[] arr, int n, int i) {
25     }
26
27     static void heapSort(int[] arr) {
28
29         int n = arr.length;
30
31         for (int i = n / 2 - 1; i >= 0; i--)
32             heapify(arr, n, i);
33
34         for (int i = n - 1; i > 0; i--) {
35
36             int temp = arr[0];
37             arr[0] = arr[i];
38             arr[i] = temp;
39
40             heapify(arr, i, 0);
41         }
42     }
43
44     public static void main(String[] args) {
45
46         Scanner sc = new Scanner(System.in);
47
48         int n = sc.nextInt();
49
50         int[] arr = new int[n];
51         int sum = 0;
52
53         for (int i = 0; i < n; i++) {
54             arr[i] = sc.nextInt();
55             sum += arr[i];
56         }
57
58         heapSort(arr);
59
60         System.out.print("Sorted: ");
61         for (int x : arr)
62             System.out.print(x + " ");
63     }
}
```

The IDE interface includes a menu bar (File, Edit, Selection, View, Go, Run, Terminal, Help), a toolbar, and a sidebar with icons for Explorer, Search, Run and Debug, Source Control, Extensions, and Settings. The status bar at the bottom shows 'Ln 87, Col 2', 'Spaces: 4', 'UTF-8', 'CRLF', 'Java', and icons for Go Live and Prettier.

